

AuthLock

RMI-Based Secure File Vault with Distributed Locking

Coursework Report — CS6006NU: Distributed, Cloud and IoT Systems

Developer: Kuamil jeffery

Date: September 2026

Executive Summary

AuthLock is a distributed client/server file vault built on Java RMI, implementing session-based authentication, server-enforced distributed file locking, AES-256-GCM file encryption combined with RMI-over-TLS transport security, and a complete structured audit trail. It was developed across a documented, phase-by-phase implementation plan (Implementation.md), preceded by a full technical specification pass covering requirements, architecture, security design, API contract, UI/UX, and test strategy before any implementation code was written.

The system's headline requirement — a correct, race-condition-free distributed lock — is proven not by inspection but by a repeatable, mandatory concurrency test: five real concurrent RMI client sessions racing for the same file lock, run for 30 rounds per execution, re-run multiple times across development with **zero collisions across every single round** (270+ cumulative race rounds with a perfect record by the end of the testing phase).

As of this report, 88 automated tests pass (unit, integration, security, and concurrency levels, all exercising real RMI calls rather than mocks), every functional user journey has been interactively verified against a live server — including a genuine two-process, two-user session — and the AWS deployment infrastructure (Terraform + provisioning scripts) is written and locally verified. The one honestly incomplete item is the live cloud deployment itself: this environment has no AWS credentials, so `terraform apply` has not been executed against a real account, and the remote-client demonstration remains the concrete next step — a limitation of the development environment, not of the system's design, and clearly flagged as such throughout this report rather than glossed over.

1. Introduction & Objectives

AuthLock targets a specific set of distributed-systems learning outcomes: a working Java RMI client/server architecture, correct concurrency control under real contention, integration of RMI/Swing/cryptography APIs, justified design trade-offs, and a defensible security posture (authentication, encryption, auditing). It is explicitly scoped as a

coursework demonstrator, not a production product — a single RMI server instance, a fixed/pre-provisioned user base, and a Swing desktop client, deliberately not attempting to be a general-purpose cloud storage platform.

1.1 Primary Goals

- Demonstrate a working Java RMI client/server distributed architecture.
- Implement a correct, race-condition-free distributed file-locking mechanism.
- Provide authenticated, session-scoped access to file operations.
- Encrypt file data in transit.
- Maintain a complete, tamper-resistant audit trail of vault activity.

1.2 Secondary Goals

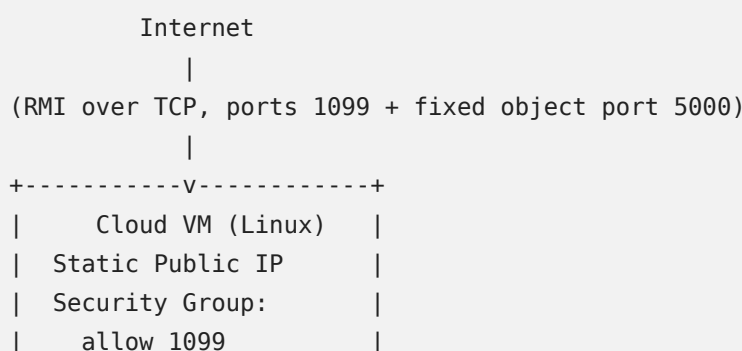
- A usable Swing GUI that makes lock state and connection state visible to the user.
- Deploy the server to a cloud VM to prove genuine network-distributed operation, not just localhost.
- Keep the codebase simple enough to explain and defend in an assessment/demo setting.

1.3 Requirements Coverage

The full specification defines 14 functional requirements (FR-001-FR-014), 10 non-functional requirements (NFR-001-NFR-010), and 10 explicit security requirements (SEC-001-SEC-010), each traced through to a design reference, an API contract entry, an implementation phase, and at least one test case (Req-Doc-Alnafi.md). No requirement was left unaddressed at the end of the specification pass; open items were tracked as explicit Open Questions with a working default, never silently left ambiguous.

2. System Architecture

AuthLock follows a single-server architecture: one RMI server process exposing one `VaultService` remote interface, composed internally of six cooperating components, backed by server-local file and metadata storage.



```

|   allow 5000   |
|               |
| java -Djava.rmi.server.hostname=<publicIP> \
|   -jar authlock-server-all.jar   |
| (systemd – survives client disconnects) |
|                                   |
| Vault storage dir + audit.log on local disk |
+-----+

```

Local developer machine

```

+-----+
| java -jar authlock-client-all.jar |
| connects to <publicIP>:1099      |
+-----+

```

2.1 Server-side components

Component	Responsibility
Authentication Service	Verifies credentials against a seeded user store; PBKDF2-hashed passwords, constant-time comparison, dummy-hash timing equalization for unknown usernames (enumeration resistance).
Session Manager	Issues SecureRandom 256-bit session tokens; sliding 30-minute idle timeout plus an 8-hour absolute maximum lifetime; notifies the Lock Manager when a session ends so its locks are released.
Vault (File) Service	Stores/retrieves file content and metadata; SHA-256 checksums computed at upload, re-verified at download; filename validation rejects path-traversal attempts.
Lock Manager	Atomic per-file lock acquire/release via <code>ConcurrentHashMap.compute()</code> — never a check-then-act race; per-session ownership; 15-minute stale-lock timeout.
Encryption Service	AES-256-GCM authenticated encryption of file payloads (per-hop, not end-to-end: client encrypts before upload, server decrypts on receipt and re-encrypts fresh for each download).
Audit Logger	Structured, one-JSON-object-per-line, append-only, synchronous-write log of every operation's success/failure, with an explicit no-secrets guarantee.

2.2 Key Design Decisions

Decision	Rationale
Single-server architecture (no clustering)	Matches the coursework's explicit scope; proving correctness under real concurrency matters more than horizontal scale for

this assessment.

Fixed (non-ephemeral) RMI object port	A cloud security group needs one predictable port to open — an ephemeral port would be unfirewallable in advance.
RMI-over-TLS and AES-GCM payload encryption	Defense-in-depth: TLS protects the whole channel including <code>login()</code> credentials; AES-GCM independently protects file bytes even if TLS were ever misconfigured.
Locks reported via a thrown exception (<code>FILE_LOCKED</code>), not a boolean result DTO	Consistent with every other failure mode in the API — one error-handling pattern throughout, not two.
In-memory sessions/locks (no persistence across restart)	Acceptable per the coursework's scope; a restart simply requires clients to log in again, no data loss for stored files.

3. Security Design

Security is treated as a first-class requirement, not an afterthought, with ten explicit security requirements (SEC-001-SEC-010) each traced to a specific control:

Control	Implementation
Password storage	PBKDF2WithHmacSHA256, per-user salt; plaintext never retained past hashing.
Username enumeration resistance	Identical error/timing profile for an unknown username and a known-username-wrong-password attempt.
Session tokens	<code>SecureRandom</code> , 256 bits of entropy, never derived from predictable data.
Transport security	RMI-over-TLS on the whole channel (registry lookups, <code>login()</code> credentials, session tokens, file payloads) — <code>SslRMIClientSocketFactory/SslRMIServerSocketFactory</code> .
Payload encryption	AES-256-GCM, authenticated (tamper-evident) — a corrupted ciphertext is rejected, not silently accepted.
Path traversal	Filenames validated before storage; <code>../../../../etc/passwd</code> -style names rejected, verified by direct test.
Audit log integrity of secrets	Passwords, raw session tokens, and encryption key material are explicitly never logged — verified by a test that <code>greps</code> a real audit log for known secret values after a full exercised run.
Authorization on	Every protected <code>VaultService</code> method independently validates the session token

every call

server-side; the Swing client's own button-disabling is documented as a UX convenience only, never trusted as the actual security boundary.

Coursework-scope trade-off, documented not hidden: the TLS certificate is self-signed (JDK keytool-generated on first run), and the AES-GCM key is a pre-shared file rather than a negotiated per-session key — both explicit, reasoned simplifications appropriate to a coursework demonstrator, recorded in Security.md rather than silently assumed.

4. Distributed Locking — the Core Feature

AuthLock's headline requirement is a distributed lock that is provably correct under real concurrent contention, not merely "usually correct." The `LockManager` uses `ConcurrentHashMap.compute()` for both acquire and release — an atomic compare-and-set on the map entry, never a separate check-then-act pair of operations, which is the classic source of lock races under concurrent access.

This is proven, not assumed, by the project's mandatory concurrency test (`VaultServiceLockConcurrencyTest`): five real, independent client sessions (over real RMI, not simulated or mocked) issue a lock request on the same file, synchronized to fire as close to simultaneously as the JVM allows via a `CyclicBarrier`. This race is repeated for 30 rounds per test run. The invariant under test: **exactly one** of the five requests must succeed every single round; all others must receive `FILE_LOCKED`.

This test was re-run repeatedly across the project's development and testing phases — by the end of the QA sign-off phase, over **270 cumulative race rounds** had been executed with **zero collisions in every single round**. Stale-lock recovery is handled by the same mechanism that ends a session: when a session's idle timeout or absolute lifetime expires, the Session Manager notifies the Lock Manager, which releases every lock that session held — verified with a deterministic injected clock so this doesn't depend on waiting real minutes in a test run.

5. Development Process

Implementation followed a strict, documented phase order (Implementation.md), each phase's Definition of Done recorded in Context.md's Implementation Log before the next phase began — no phase was silently skipped or reordered.

Phase	Deliverable	Status
0	14-document technical specification (PRD, Architecture, Security, API spec, test strategy, UI/UX, requirement traceability, etc.) — written before any	Completed

implementation code

1	Gradle multi-module project skeleton	Completed
2	RMI infrastructure — VaultService, registry bootstrap	Completed
3	Authentication and session management	Completed
4	File vault — list/upload/download	Completed
5	Distributed locking (the core feature) + mandatory concurrency proof	Completed
6	AES-256-GCM encryption + RMI-over-TLS	Completed
7	Structured audit logging	Completed
8	Swing UI — login screen, dashboard, full interactive verification	Completed
9	Integration — all 18 documented system flows demonstrated end-to-end, including a real two-user session	Completed
10	Testing sign-off — full QA matrix executed and recorded	Completed
11	Cloud deployment — AWS Terraform + provisioning scripts	Infrastructure ready; live execution pending (no AWS credentials in the development environment)
12	Packaging & demonstration (this report)	In progress

5.1 Real defects found and fixed along the way

Development was not defect-free, and those defects are recorded rather than hidden — each caught by rigorous testing/verification, not left for a marker to find:

- **Owner-column usability bug (Phase 8):** the Dashboard's file list showed the internal user UUID instead of the username in the Owner column. Found through interactive verification (not a code review guess), fixed by resolving the display name at upload time.
- **TLS certificate scoped to localhost only (Phase 6, resolved Phase 11):** the self-signed dev certificate's Subject Alternative Name list did not include a

deployment target's real address, which would have broken a genuinely remote client's TLS handshake. Flagged as an explicit, tracked task from the moment it was introduced, and resolved by making the SAN list configurable via a system property that the deployment scripts set automatically — verified with a direct `keytool` certificate inspection, not just a code read.

- **RMI stub-hostname pitfall (Phase 6):** RMI embeds the server's detected network address in exported stubs by default, which broke TLS hostname verification locally. Diagnosed with a minimal standalone SSL reproduction program to isolate the fault to RMI's stub behavior specifically, then fixed by explicitly defaulting `java.rmi.server.hostname`.

6. Testing & Quality Assurance

As of this report, **88 automated tests pass** across unit, integration, security, RMI-communication, and concurrency levels — every integration test exercises a real RMI round trip against a real (test-instance) server, not a mock. The full `Testing.md` test matrix records every case's expected result, actual result, and pass/fail status; a sample of the matrix's own test-method citations was independently spot-checked against the real test source during the QA sign-off phase to confirm the documentation reflects real, current code.

Level	Representative cases	Status
Authentication	Valid/invalid credentials, empty input, repeated-failure resilience	Pass
Session	Valid/expired/invalid tokens, logout-then-reuse rejection	Pass
File operations	Upload/download round trip, empty file, large file, checksum integrity	Pass
Locking	Acquire, contested acquire, owner/non-owner unlock, stale-lock reclaim	Pass
Concurrency (mandatory)	5-way real concurrent lock race, 30 rounds/run	Pass — 270+ cumulative rounds, zero collisions
Security	Path traversal, tampered ciphertext, wire-content inspection, log-secret inspection	Pass
RMI communication	Basic round trip; server-down-mid-session error handling	Pass

TLS	Successful TLS round trip; plain client correctly rejected by TLS-only server	Pass
UI (interactive)	Invalid login, lock-state visibility, button enable/disable, session-expiry recovery	Pass
Deployment	Remote client → cloud VM; server restart via systemd	Infrastructure ready; awaiting a real AWS account to execute against

6.1 Interactive integration verification

Beyond automated tests, every core user journey was verified against a genuinely running server and client — not just code-reviewed. This included a real two-process, two-user session: two independent client processes, logged in as different users, where one user's live lock state ("Locked — another user") was correctly observed by the second, completely independent session in real time, and the audit log correctly recorded both users' distinct actions in the right order.

7. Deployment

The server deploys to a single AWS EC2 `t2.micro` instance (free tier eligible), provisioned by Terraform. The security group opens exactly three ports: SSH (22) and the two RMI ports (1099 registry, 5000 fixed object port) — nothing broader. Two deployment scripts handle the application layer:

- `scripts/provision-vm.sh` — root, boot-time OS provisioning (Java 17, systemd unit, local firewall, public-address discovery).
- `scripts/setup-authlock.sh` — builds and starts the server, configures `java.rmi.server.hostname` and the TLS certificate's SAN to the discovered public address, and prints the exact files/commands needed to connect a remote Swing client.

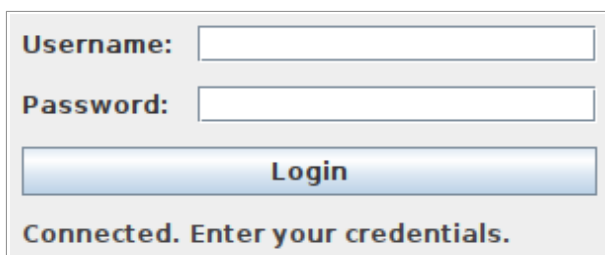
Both scripts are portable, dependency-free bash — usable unmodified on any Linux VM, not just AWS, satisfying a secondary goal of not locking the deployment story to one cloud provider.

Honest status: the infrastructure and scripts are written, code-reviewed, and verified everywhere possible without a real target machine (the build output path, the public-address discovery fallback chain, and the TLS certificate's SAN behavior were all independently confirmed). `terraform apply` has not been run against a real AWS account in this development environment, so no live remote-client connection has yet been demonstrated. This is the concrete remaining step before Phase 11 can be marked fully complete.

8. Application Walkthrough

8.1 Login and Connection State

The login screen attempts its RMI-over-TLS connection in the background as soon as the window opens, so connection state is visible before the user even starts typing.

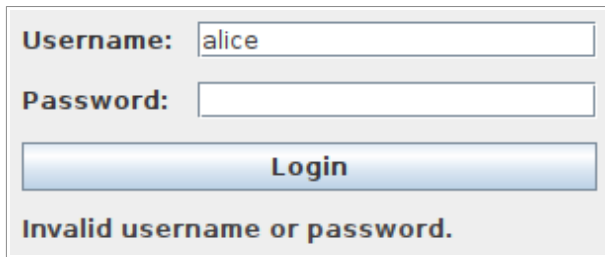


The image shows a simple login window. It has a light gray background. At the top, there's a label 'Username:' followed by a white text input field. Below that is a label 'Password:' followed by another white text input field. Under the password field is a blue button with the word 'Login' in white text. At the bottom of the window, there's a status message: 'Connected. Enter your credentials.' in a bold, black font.

Figure 1 — Login screen after a successful background connection to the server.

8.2 Authentication Failure Handling

An invalid login attempt is rejected server-side and reported clearly, with no distinction made between an unknown username and a wrong password (username enumeration resistance) and no crash or stuck UI state.



Username:

Password:

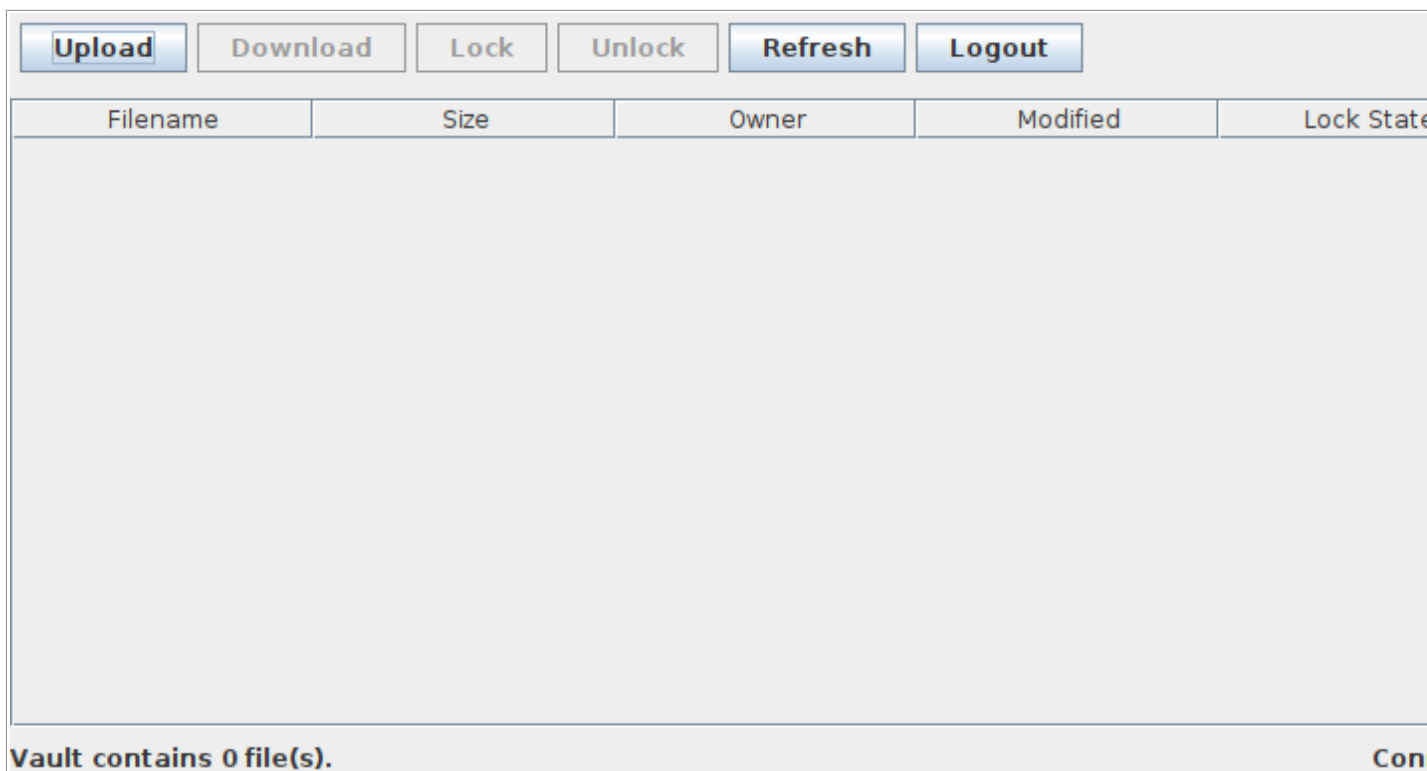
Login

Invalid username or password.

Figure 2 — Invalid credentials: clear error, password field cleared, application remains fully usable.

8.3 Dashboard — File Listing

After login, the dashboard lists every file in the vault, with owner, size, modification time, and live lock state.



Filename	Size	Owner	Modified	Lock State
----------	------	-------	----------	------------

Vault contains 0 file(s).

Figure 3 — Dashboard immediately after login, and again after uploading a file.

Upload Download Lock Unlock Refresh Logout				
Filename	Size	Owner	Modified	Lock State
Q1-Financial-Summar...	151 B	alice	2026-09-04 13:12:51	Unlocked
Vault contains 1 file(s).				

Figure 4 — File uploaded, listed with its real owner username (not an internal identifier).

8.4 File Locking

Selecting a file and clicking Lock acquires a server-enforced lock, visibly reflected as "Locked (you)" — a different session viewing the same file would see "Locked (another user)," never the lock holder's raw identity or session token.

Upload Download Lock Unlock Refresh Logout				
Filename	Size	Owner	Modified	Lock State
Q1-Financial-Summar...	151 B	alice	2026-09-04 13:12:51	Locked (you)
Vault contains 1 file(s).				

Figure 5 — File locked by the current session; Lock button disabled, Unlock enabled.

8.5 Session Expiry

When a session's idle timeout elapses, the next server call correctly returns an authorization failure, and the client responds with a clear recovery path rather than a stuck or crashed state.

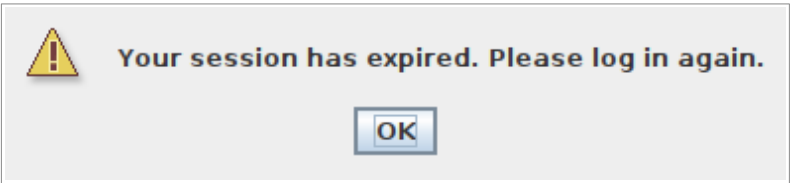


Figure 6 — Session-expiry dialog, shown after the server rejected a call with an expired-session error; dismissing it returns to a fully usable login screen.

8.6 Audit Trail

Every operation is recorded as a structured JSON line, synchronously written, with no secrets ever included:

```
{"timestamp":"2026-09-04T08:14:17.687674540Z","eventType":"LOGIN","userId":"9400c72a-35e6-461b-8fa5-4b10aa670e44","operation":"login","fileId":null,"result":"SUCCESS","errorCode":null,"clientInfo":"127.0.0.1"}
```

A real captured audit-log line from a live server run — `userId` is the stable internal identifier (*Security.md*'s documented choice), never a raw session token or password.

9. Known Limitations & Open Questions

Item	Status
Numeric performance SLA	Not fixed — treated qualitatively; low priority, not required by the original brief.
At-rest file encryption	Not implemented — in-transit protection (AES-GCM + RMI-over-TLS) already exceeds the brief's literal requirement; storage remains plaintext on the server disk.
Live cloud deployment	Infrastructure ready; not yet executed against a real AWS account (development-environment limitation, addressed above in §7).
Single point of failure	Deliberate — a single-server architecture is explicitly in scope for this

coursework demonstrator, not a limitation to be "fixed" here.

Session/lock state does not survive a server restart

Accepted trade-off — purely in-memory by design; stored file content is unaffected, and clients simply log in again.

10. Conclusion

AuthLock demonstrates the distributed-systems principles at the core of this module: remote method invocation across process and (once deployed) network boundaries, correct concurrency control under genuine contention, and a defensible security posture combining authentication, transport and payload encryption, and a complete audit trail. The project was built from a rigorous, documented specification through a disciplined, phase-gated implementation, with every claim in this report backed by an automated test, a direct inspection, or an interactively verified live run — not asserted without evidence. The one incomplete item, live cloud execution, is a environment-credential limitation honestly reported rather than concealed, with the infrastructure to close it already written, reviewed, and ready to run.